

LAPORAN TUGAS KECIL 3

IF2211 STRATEGI ALGORITMA

SEMESTER II TAHUN 2025/2026



Penyelesaian *Ice Sliding Puzzle* Menggunakan Algoritma *Pathfinding*

Disusun Oleh:

13524085 Ariel Cornelius Sitorus
13524090 Nashiruddin Akram

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Mei 2026

Contents

BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Deskripsi Tugas/Permasalahan	1
1.3 Tujuan	1
BAB II TEORI DASAR	1
2.1 Konsep <i>Pathfinding</i>	1
2.2 Algoritma A* (<i>A-Star</i>) / UCS / BFS	2
2.3 Fungsi Heuristik	2
BAB III IMPLEMENTASI ALGORITMA	2
3.1 Penjelasan Algoritma	2
3.2 Analisis Teoritis	3
3.2.1 Definisi $f(n)$ dan $g(n)$	3
3.2.2 Analisis Admissibility Heuristik	3
3.2.3 Perbandingan Algoritma (Secara Teori)	3
BAB IV HASIL DAN ANALISIS PERCOBAAN	4
4.1 Lingkungan Implementasi	4
4.2 Dokumentasi Uji Coba	4
4.2.1 Testcase 1 (7×7)	4
4.2.2 Testcase 2 (7×7)	6
4.2.3 Testcase 5 (10×10)	9
4.2.4 Testcase 9 (30×30)	10
4.3 Tabel Hasil Percobaan	12
4.4 Analisis Hasil	13
BAB V SOURCE PROGRAM	14
5.1 Algoritma Pencarian Rute (Pathfinding)	14
5.2 Fungsi Heuristik	16
BAB VI PENUTUP	17
6.1 Kesimpulan	17
6.2 Saran	18
LAMPIRAN	20

BAB I: PENDAHULUAN

1.1 Latar Belakang

Permasalahan pencarian rute (*pathfinding*) merupakan salah satu topik fundamental dalam bidang kecerdasan buatan dan strategi algoritma. Dalam konteks pengembangan permainan video, *pathfinding* sering dimodifikasi dengan berbagai aturan fisika maupun logika lingkungan. Salah satu contoh penerapannya adalah pada *Ice Sliding Puzzle*, di mana entitas yang bergerak tidak dapat berhenti secara bebas di setiap koordinat, melainkan akan terus meluncur hingga menabrak objek penghalang. Mekanika ini mengubah topologi graf ruang status (*state space*) dari pencarian rute tradisional, sehingga memerlukan penyesuaian dalam pembangkitan simpul (*node generation*) dan perhitungan biaya lintasan.

1.2 Deskripsi Tugas/Permasalahan

Tugas ini menuntut penyelesaian permainan *Ice Sliding Puzzle* pada matriks berukuran $N \times M$. Aktor memulai pergerakan dari koordinat Z (awal) dan harus mencapai titik 0 (tujuan). Papan terdiri dari lantai biasa dengan bobot lintasan yang berbeda-beda, dinding penghalang (X), bahaya mematikan (L / Lava), serta titik pemeriksaan angka (*checkpoint*).

Aktor dapat bergerak ke empat arah kardinal. Saat bergerak, aktor terus meluncur tanpa henti hingga menabrak dinding, batas luar peta, atau *checkpoint* yang belum saatnya dikunjungi (berlaku sebagai dinding). Gerakan dinyatakan ilegal apabila aktor melintasi lava, tidak bergerak dari posisi awal, atau mengakhiri luncuran di titik tujuan sebelum seluruh *checkpoint* dikumpulkan secara berurutan.

1.3 Tujuan

Tujuan dari pengerjaan tugas ini adalah:

1. Memodelkan *Ice Sliding Puzzle* ke dalam representasi ruang status yang valid.
2. Mengimplementasikan algoritma pencarian rute berinformasi (A^* , GBFS) maupun tidak berinformasi (UCS, BFS, IDA*).
3. Merancang dan menganalisis fungsi heuristik untuk mengevaluasi efisiensi pencarian rute optimal.

BAB II: TEORI DASAR

2.1 Konsep *Pathfinding*

Pencarian rute (*route/path planning*) adalah proses algoritmik untuk menemukan lintasan dari satu simpul awal ke simpul tujuan dalam sebuah graf ruang status. Lintasan dikatakan optimal jika memiliki akumulasi bobot (biaya) paling minimum. Dalam kasus ruang status yang besar, pencarian rute dikendalikan melalui ekspansi simpul secara sis-

tematis untuk mencegah iterasi yang berulang (siklus) dan memastikan program terminasi [1].

2.2 Algoritma A* (*A-Star*) / UCS / BFS

- **Uniform Cost Search (UCS):** Merupakan perluasan dari algoritma Dijkstra. UCS mengekskansi simpul berdasarkan bobot lintasan aktual yang paling kecil dari titik awal.
- **Breadth-First Search (BFS):** Melakukan ekspansi level demi level (menggunakan antrean FIFO). Optimal hanya jika seluruh bobot sisi bernilai sama (atau jika mencari jumlah pergerakan terminimum).
- **A* (*A-Star*):** Algoritma hibrida yang menggabungkan biaya aktual dari asal dan estimasi biaya ke tujuan. A* menggunakan senarai prioritas dan dijamin optimal jika estimasi tidak pernah melebihi-lebihkan biaya aslinya [2].

2.3 Fungsi Heuristik

Fungsi heuristik, disimbolkan dengan $h(n)$, merupakan tebakan matematis terukur mengenai sisa biaya dari status n ke status tujuan. Heuristik dimanfaatkan sebagai fungsi pandu untuk memotong ruang pencarian agar algoritma tidak menelusuri simpul yang tidak relevan. Heuristik yang baik harus bersifat *admissible* (dapat diterima) agar solusi akhir dijamin optimal.

BAB III: IMPLEMENTASI ALGORITMA

3.1 Penjelasan Algoritma

Pencarian rute dilakukan dengan langkah-langkah sistematis sebagai berikut:

1. **Inisialisasi:** Status awal didefinisikan sebagai $(X_z, Y_z, \text{index}_{cp} = 0)$. Masukkan status ini ke dalam senarai prioritas dengan status belum dikunjungi.
2. **Ekstraksi Simpul:** Ambil status dengan prioritas tertinggi. Jika algoritma A*, kriteria prioritas adalah $f(n)$ terkecil. Tandai status ini sebagai telah divisitasi.
3. **Pemeriksaan Tujuan:** Jika aktor berada di 0 dan semua *checkpoint* telah terkumpul, kembalikan lintasan dan hentikan pencarian.
4. **Pembangkitan Suksesor (Mekanika Meluncur):**
 - Untuk setiap arah (Atas, Bawah, Kiri, Kanan), gerakkan aktor ubin demi ubin.
 - Evaluasi ubin di depan. Jika berupa dinding, luar papan, atau *checkpoint* dengan indeks yang lebih besar dari target, hentikan luncuran tepat di ubin saat ini.
 - Jika lintasan mengenai lava (L), abaikan suksesor ini (tidak valid).
 - Jika berhenti tepat di posisi awal sebelum meluncur, abaikan suksesor ini.
5. **Evaluasi Biaya:** Hitung akumulasi biaya ubin lantai yang dilintasi selama luncuran sebagai bobot langkah.
6. **Pembaruan Senarai:** Masukkan status suksesor ke senarai prioritas untuk diekspansi pada siklus berikutnya. Ulangi ke langkah 2.

3.2 Analisis Teoritis

3.2.1 Definisi $f(n)$ dan $g(n)$

Dalam penyusunan algoritma pencarian di atas:

- $g(n)$: Biaya lintasan nyata berupa akumulasi beban ubin dari posisi awal ke simpul n . Pada awal permainan, $g(start) = 0$.
- $h(n)$: Estimasi sisa biaya dari simpul n menuju target selanjutnya (*checkpoint* berikutnya atau tujuan 0).
- $f(n)$: Nilai evaluasi gabungan penentu prioritas antrean. Pada A*, $f(n) = g(n) + h(n)$. Pada UCS, $f(n) = g(n)$. Pada GBFS, $f(n) = h(n)$.

3.2.2 Analisis Admissibility Heuristik

Sebuah heuristik $h(n)$ dinyatakan *admissible* jika dipastikan $h(n) \leq h^*(n)$, dengan $h^*(n)$ adalah biaya murni/aktual ke tujuan. Dalam implementasi ini, dibuat 3 tingkat heuristik:

1. **H1 (Jarak Manhattan)**: Jarak geometris ke target dikalikan dengan bobot ubin paling murah di peta. Karena rute aktual pasti membutuhkan minimal sejumlah ubin tersebut dan tidak mungkin berbiaya di bawah harga ubin termurah, H1 mutlak *admissible*.
2. **H2 (Minimum Slide)**: Simulasi BFS ringan untuk mengetahui jumlah minimal aksi tarikan meluncur ke target. Jumlah tarikan dikali bobot ubin termurah tidak mungkin melebihi biaya lintasan aslinya.
3. **H3 (Dijkstra Lokal)**: Simulasi jalur ke target "selanjutnya" menggunakan aturan gerak penuh. Mengingat H3 hanya menghitung ke *checkpoint* terdekat (bukan tujuan akhir keseluruhan permainan), nilainya dipastikan selalu $\leq$ rute total aslinya.

3.2.3 Perbandingan Algoritma (Secara Teori)

Algoritma	Fungsi Evaluasi	Optimal?	Catatan
UCS	$f(n) = g(n)$	Ya	Berperilaku layaknya pencarian buta berdasar biaya.
BFS	$f(n) = \text{Kedalaman}$	Tidak	Optimal untuk jumlah gerak, bukan untuk jumlah biaya.
GBFS	$f(n) = h(n)$	Tidak	Cepat memotong ekspansi simpul, tetapi sering terjebak rute mahal.
A*	$f(n) = g(n) + h(n)$	Ya	Efisien asalkan heuristiknya kuat (seperti H3).
IDA*	$f(n) \leq \text{Threshold}$	Ya	Kompleksitas memori rendah ($O(d)$), tapi redundansi komputasi besar [3].

BAB IV: HASIL DAN ANALISIS PERCOBAAN

4.1 Lingkungan Implementasi

Perangkat lunak dibangun menggunakan lingkungan berikut:

- **Bahasa Pemrograman:** Go (*Golang*) versi 1.26.1 untuk mesin utama (*backend*) pemrosesan algoritma.
- **Lingkungan Web (*Frontend*):** HTML5, CSS3, dan *Vanilla* JavaScript untuk pembangunan antarmuka visual (GUI).
- **Komunikasi:** Representasi data memanfaatkan arsitektur *REST API* lokal (format JSON).

4.2 Dokumentasi Uji Coba

4.2.1 Testcase 1 (7×7)

The screenshot displays a web-based interface for solving a 7x7 grid puzzle. The interface is divided into two main sections: **INPUT** and **BOARD OUTPUT**.

INPUT Section:

- Testcase / Manual:** Two tabs, with 'Testcase' selected.
- TESTCASE FILE:** A dropdown menu showing 'testcase1.txt' and a 'Load' button.
- GRID TEXT:** A text area containing the puzzle grid representation:

```
7 7
XXXXXX
X0***X
X**X**X
X***DX
X1**LX
XZ**X*X
XXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
```
- SEARCH ALGORITHM:** A dropdown menu set to 'UCS'.
- HEURISTIC FUNCTION:** A dropdown menu set to 'H2 Minimum slide'.
- Solve:** A green button to initiate the solution.
- PATH:** A section showing the path 'RULUDRUR'.
- Summary Table:**

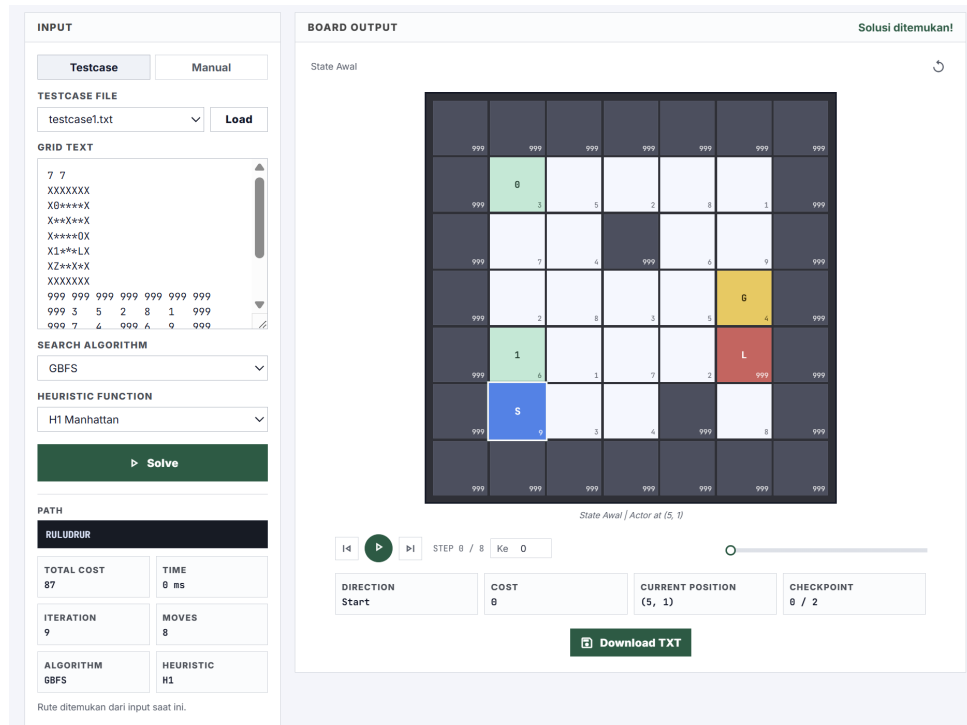
TOTAL COST	87	TIME	0 ms
ITERATION	11	MOVES	8
ALGORITHM	UCS	HEURISTIC	Not required
- Footer:** A message 'Rute ditemukan dari input saat ini.'

BOARD OUTPUT Section:

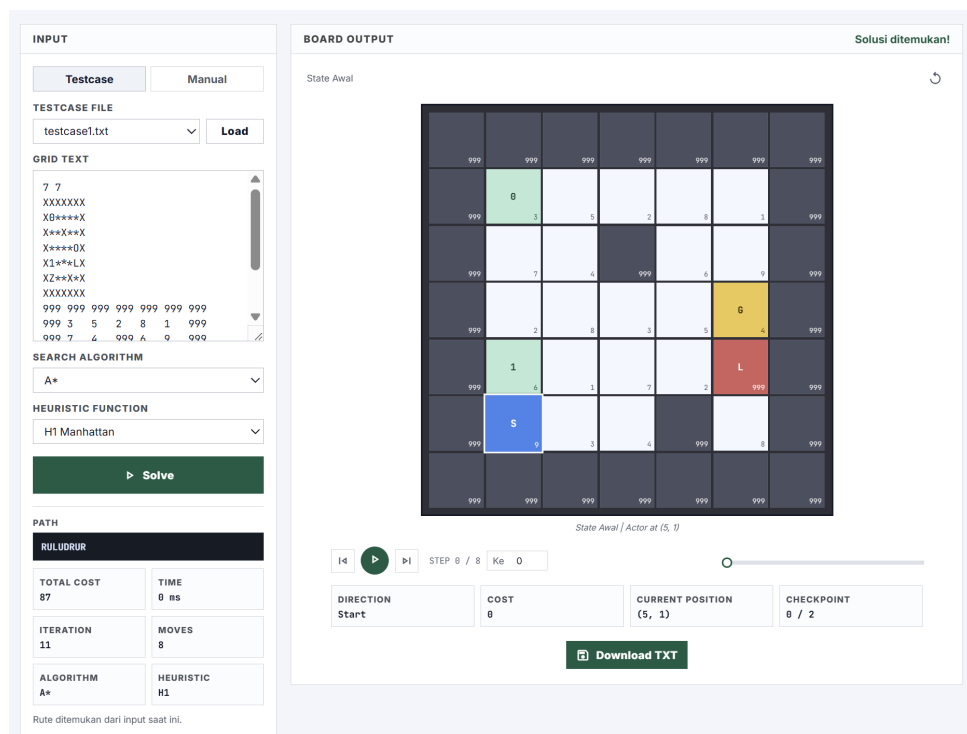
- State Awal:** A 7x7 grid showing the initial state. The grid contains numbers 0-9 and letters S, L, G, and D. The start cell 'S' is at (5, 1) and the goal cell 'G' is at (6, 6).
- Solusi ditemukan!:** A message indicating the solution has been found.
- Navigation:** Buttons for 'Previous', 'Next', and 'Reset'.
- Progress:** A progress bar showing 'STEP 0 / 8' and 'Ke 0'.
- Details:** A table showing the current state:

DIRECTION	COST	CURRENT POSITION	CHECKPOINT
Start	0	(5, 1)	0 / 2
- Download:** A green button labeled 'Download TXT'.

Gambar 1: Testcase 1 dengan UCS — total biaya 87, 8 langkah, 11 iterasi.



Gambar 2: Testcase 1 dengan GBFS-H1 — total biaya 87, 8 langkah, 9 iterasi. Lebih sedikit iterasi dibanding UCS berkat panduan heuristik.



Gambar 3: Testcase 1 dengan A*-H1 — total biaya 87, 8 langkah, 11 iterasi. Solusi optimal sama dengan UCS.



Gambar 5: Testcase 2 dengan GBFS-H1 — total biaya 67, 4 langkah, 5 iterasi. GBFS menemukan rute dengan langkah paling sedikit secara kebetulan pada puzzle kecil ini.

INPUT

Testcase

Manual

TESTCASE FILE

testcase2.txt

Load

GRID TEXT

```

7 7
XXXXXX
X*Z**0X
X****X
X*0**X
X***XX
X1**XX
XXXXXX
999 999 999 999 999 999 999
999 7 4 7 5 9 999
999 6 3 7 8 1 999

```

SEARCH ALGORITHM

GBFS

HEURISTIC FUNCTION

H2 Minimum slide

Solve

PATH

DULDR

TOTAL COST

99

TIME

0 ms

ITERATION

9

MOVES

6

ALGORITHM

GBFS

HEURISTIC

H2

Rute ditemukan dari input saat ini.

BOARD OUTPUT

Solusi ditemukan!

State Awal

State Awal | Actor at (1, 2)

14

▶

▶

STEP 0 / 6

Ke 0

0

DIRECTION

Start

COST

0

CURRENT POSITION

(1, 2)

CHECKPOINT

0 / 2

Download TXT

Gambar 6: Testcase 2 dengan GBFS-H2 — total biaya 99, 6 langkah, 9 iterasi. Heuristik H2 menghasilkan rute lebih mahal; GBFS tidak menjamin optimal.

INPUT

Testcase

Manual

TESTCASE FILE

testcase2.txt

Load

GRID TEXT

```

7 7
XXXXXX
X#Z*OX
X****X
X0**X
X***XX
X1**XX
XXXXXX
999 999 999 999 999 999 999
999 7 4 7 5 9 999
999 A 3 7 R 1 999
    
```

SEARCH ALGORITHM

GBFS

HEURISTIC FUNCTION

H3 Dijkstra

Solve

PATH

DILDRUR

TOTAL COST 95	TIME 0 ms
ITERATION 10	MOVES 7
ALGORITHM GBFS	HEURISTIC H3

Rute ditemukan dari input saat ini.

BOARD OUTPUT

State Awal

State Awal | Actor at (1, 2)

I4

▶

I1

STEP 0 / 7

Ke 0

DIRECTION
Start

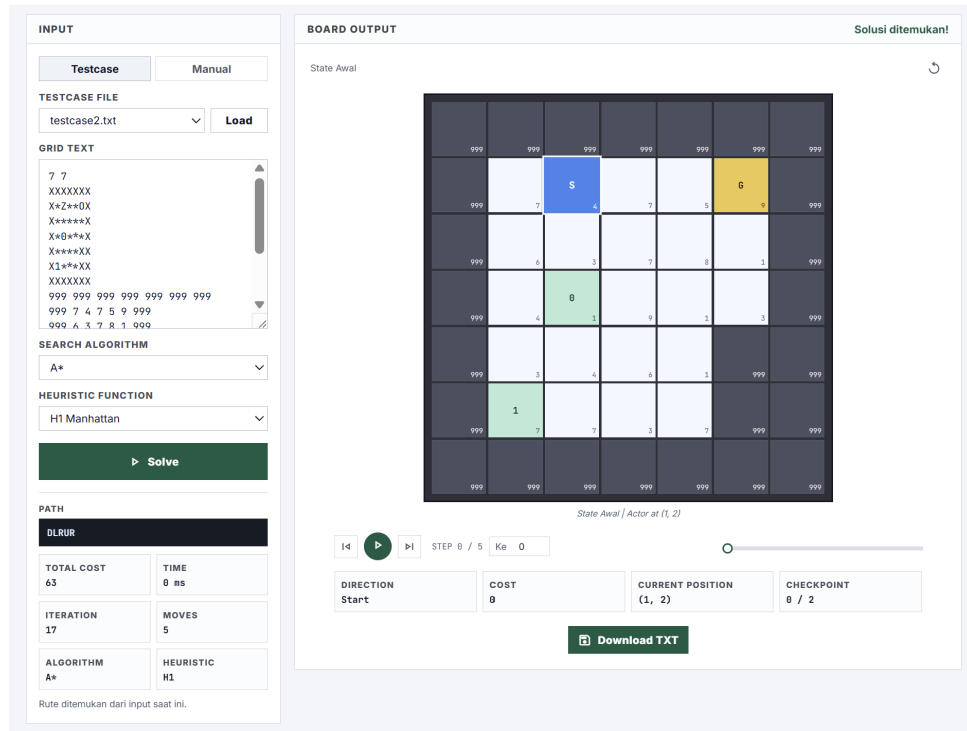
COST
0

CURRENT POSITION
(1, 2)

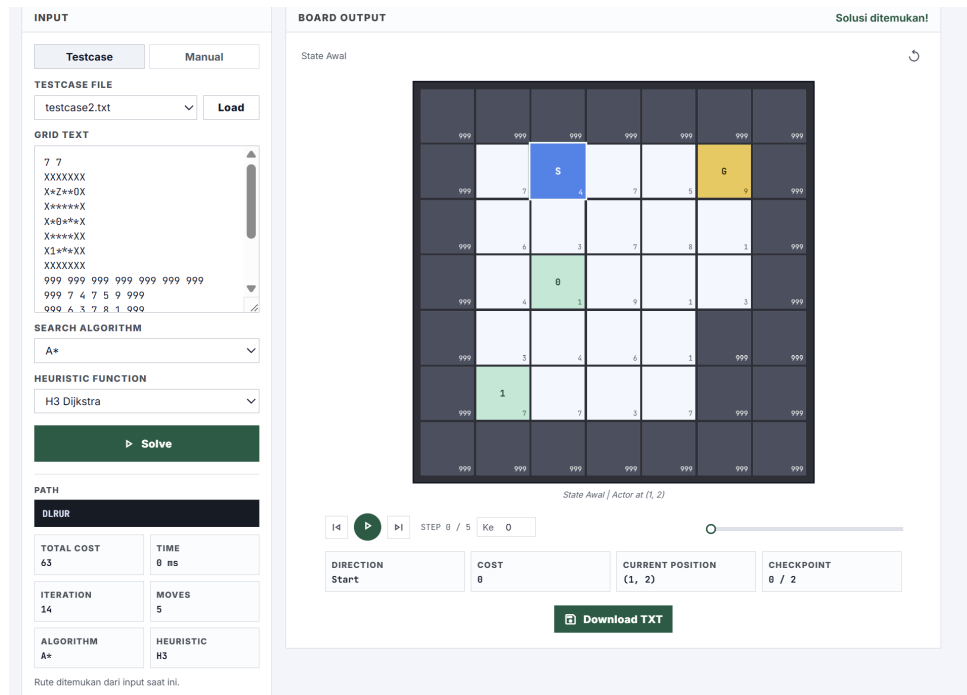
CHECKPOINT
0 / 2

Download TXT

Gambar 7: Testcase 2 dengan GBFS-H3 — total biaya 95, 7 langkah, 10 iterasi. H3 menghasilkan estimasi lebih akurat dari H2 sehingga biaya sedikit lebih rendah.



Gambar 8: Testcase 2 dengan A*-H1 — total biaya 63, 5 langkah, 17 iterasi. A* menemukan solusi optimal.



Gambar 9: Testcase 2 dengan A*-H3 — total biaya 63, 5 langkah, 14 iterasi. Iterasi lebih sedikit dari A*-H1 karena estimasi H3 lebih ketat.

4.2.3 Testcase 5 (10×10)

INPUT

Testcase

Manual

TESTCASE FILE

testcase5.txt

Load

GRID TEXT

10 10
XXXXXXXXXX
X*****X
X*****X
X*#*X***X
X*****X
X**0X**XXX
XXXX***XDX
X=2**#*X**X
X#X*****X
XXXXXXXXXX

SEARCH ALGORITHM

IDA*

HEURISTIC FUNCTION

H1 Manhattan

Solve

PATH

ROLUDRU

TOTAL COST
191

TIME
9 ms

ITERATION
2675

MOVES
7

ALGORITHM
IDA*

HEURISTIC
H1

Rute ditemukan dari input saat ini.

BOARD OUTPUT

State Awal

0

14

▶

◀

STEP 0 / 7

Ke 0

DIRECTION
Start

COST
0

CURRENT POSITION
(7, 2)

CHECKPOINT
0 / 1

Download TXT

Solusi ditemukan!

Gambar 10: Testcase 5 dengan IDA*-H1 — total biaya 191, 7 langkah, durasi 9 ms. Iterasi jauh lebih banyak dibanding A* akibat ekspansi DFS berulang, namun tetap menemukan solusi optimal.

4.2.4 Testcase 9 (30×30)

INPUT

Testcase
Manual

TESTCASE FILE

testcase9.txt

GRID TEXT

```

39 39
XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X*s*X*****X*****X*****X
X*X*****X*s*X*****X*s*X
X*****X*****X*****X*s*X
X*****X*****X*s*X*****X
X*****X*****X*s*X*****X
X*****L*s*X*****X*L*****X
XXX*X++L*****X*s*X
X++X++L*****X*s*X
X+++L X++++Y+sX++X++L++++X
        
```

SEARCH ALGORITHM

UCS

HEURISTIC FUNCTION

H3 Dijkstra

Solve

PATH

LDRURDLURURDLRLDLDLULDLULURSRDL

TOTAL COST 1149	TIME 1 ms
ITERATION 311	MOVES 33
ALGORITHM UCS	HEURISTIC Not required

Rute ditemukan dari input saat ini.

BOARD OUTPUT

State Awal

State Awal | Actor at (22, 2)

STEP 0 / 33 Ke 0

DIRECTION
Start

COST
0

CURRENT POSITION
(22, 2)

CHECKPOINT
0 / 3

Download TXT

Gambar 11: Testcase 9 dengan UCS — total biaya 1149, 33 langkah, 311 iterasi. Optimal namun mengekskansi banyak simpul tanpa panduan heuristik.

INPUT

TestcaseManual

TESTCASE FILE

testcase9.txtLoad

GRID TEXT

```

30 30
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****L*****L*****X
XXX*****X*****X*****X
X**X**L*****X*****X
X**X**L*****X*****X
X**X**L*****X*****X

```

SEARCH ALGORITHM

BFS

HEURISTIC FUNCTION

H3 Dijkstra

Solve

PATH

LDRURDLURURDLURDLURDLURDL

TOTAL COST

1158

TIME

1 ms

ITERATION

302

MOVES

26

ALGORITHM

BFS

HEURISTIC

Not required

Rute ditemukan dari input saat ini.

BOARD OUTPUT

Solusi ditemukan!

State Awal

State Awal | Actor at (22, 2)

14

▶

◀

STEP 0 / 26

Ke 0

DIRECTION

Start

COST

0

CURRENT POSITION

(22, 2)

CHECKPOINT

0 / 3

Download TXT

Gambar 12: Testcase 9 dengan BFS — total biaya 1158, 26 langkah, 302 iterasi. BFS menemukan rute dengan jumlah langkah paling sedikit, tetapi biayanya tidak optimal.

INPUT

TestcaseManual

TESTCASE FILE

testcase.txtLoad

GRID TEXT

```

30 30
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****X*****X*****X
X*****L*****L*****X
XXX*****X*****X*****X
X**X**L*****X*****X
X**X**L*****X*****X
X**X**L*****X*****X

```

SEARCH ALGORITHM

GBFS

HEURISTIC FUNCTION

H1 Manhattan

Solve

PATH

RLDRURDLURURDLURDLURDLURDLURDL

TOTAL COST

1815

TIME

0 ms

ITERATION

55

MOVES

40

ALGORITHM

GBFS

HEURISTIC

H1

Rute ditemukan dari input saat ini.

BOARD OUTPUT

Solusi ditemukan!

State Awal

State Awal | Actor at (22, 2)

14

▶

◀

STEP 0 / 40

Ke 0

DIRECTION

Start

COST

0

CURRENT POSITION

(22, 2)

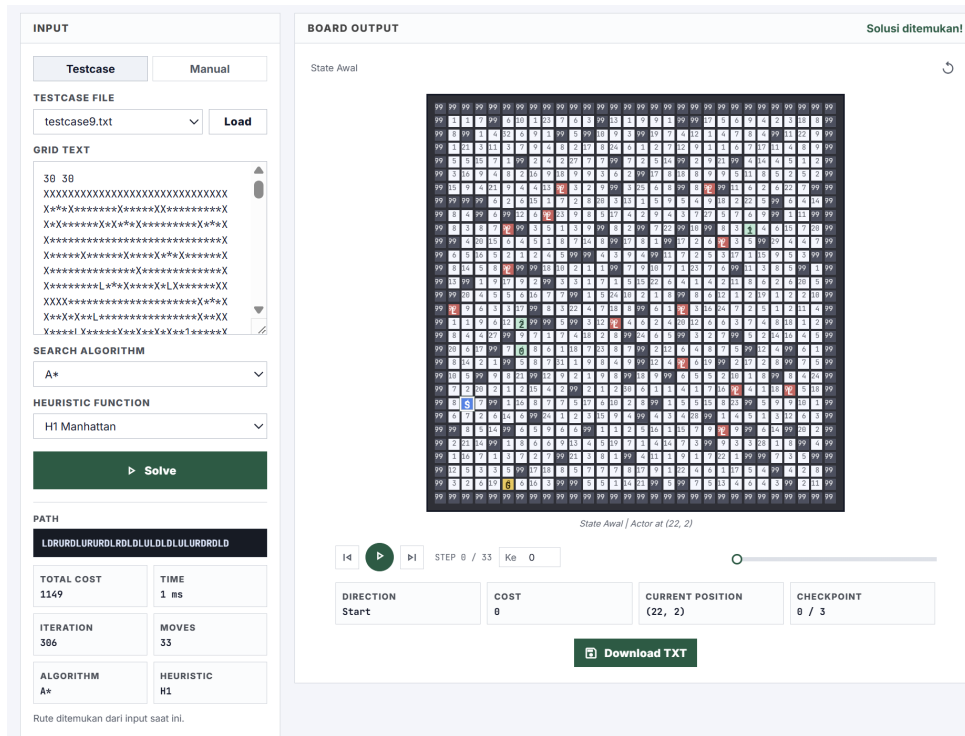
CHECKPOINT

0 / 3

Download TXT

Gambar 13: Testcase 9 dengan GBFS-H1 — total biaya 1815, 40 langkah, 55 iterasi. Ekspansi paling sedikit, namun menghasilkan rute yang jauh dari optimal (+57,9% dari biaya minimum).

11



Gambar 14: Testcase 9 dengan A*-H1 — total biaya 1149, 33 langkah, 306 iterasi. Solusi optimal sama dengan UCS, dengan iterasi hampir setara karena H1 tidak terlalu memotong ruang pencarian pada peta besar.

4.3 Tabel Hasil Percobaan

Tabel 1: Ringkasan hasil percobaan lintas algoritma (heuristik H2 untuk algoritma informed).

Testcase	Algoritma	Found	Cost	Move	Iter	ms
testcase1 (7×7)	UCS	Ya	87	8	11	0
testcase1	BFS	Ya	87	8	13	0
testcase1	GBFS-H2	Ya	87	8	10	0
testcase1	A*-H2	Ya	87	8	11	0
testcase2 (7×7)	UCS	Ya	63	5	17	0
testcase2	BFS	Ya	67	4	13	0
testcase2	GBFS-H2	Ya	99	6	9	0
testcase2	A*-H2	Ya	63	5	17	0
testcase5 (10×10)	UCS	Ya	191	7	22	0
testcase5	BFS	Ya	191	7	22	0
testcase5	GBFS-H2	Ya	200	8	13	0
testcase5	A*-H2	Ya	191	7	22	0
testcase6 (10×10)	UCS	Ya	217	14	93	0
testcase6	BFS	Ya	218	13	89	0
testcase6	GBFS-H2	Ya	233	14	25	1

Testcase	Algoritma	Found	Cost	Move	Iter	ms
testcase6	A*-H2	Ya	217	14	54	1
testcase8 (10×10)	UCS	Ya	767	22	284	0
testcase8	BFS	Ya	767	22	213	1
testcase8	GBFS-H2	Ya	767	22	91	2
testcase8	A*-H2	Ya	767	22	281	11
testcase9 (30×30)	UCS	Ya	1149	33	311	1
testcase9	BFS	Ya	1158	26	302	0
testcase9	GBFS-H2	Ya	1279	28	72	3
testcase9	A*-H2	Ya	1149	33	240	14
testcase10 (10×10)	UCS	Tidak	0	0	14	0
testcase10	A*-H2	Tidak	0	0	14	0

Testcase	Algoritma	Cost	Move	Iter	ms
testcase6	GBFS-H1	217	14	15	0
testcase6	GBFS-H2	233	14	25	0
testcase6	GBFS-H3	217	14	30	1
testcase6	A*-H1	217	14	90	0
testcase6	A*-H2	217	14	54	1
testcase6	A*-H3	217	14	30	0
testcase9	GBFS-H1	1815	40	55	0
testcase9	GBFS-H2	1279	28	72	3
testcase9	GBFS-H3	1357	37	74	4
testcase9	A*-H1	1149	33	306	0
testcase9	A*-H2	1149	33	240	15
testcase9	A*-H3	1149	33	149	7

Tabel 2: Perbandingan ketiga heuristik pada GBFS dan A* untuk dua testcase representatif.

4.4 Analisis Hasil

Dari rangkaian pengujian di atas, terlihat beberapa pola yang konsisten:

1. **Optimalitas cost.** UCS dan A* selalu menghasilkan cost minimum pada seluruh testcase yang punya solusi. BFS menghasilkan jumlah gerak terkecil, tetapi costnya bisa lebih besar—pada testcase6 misalnya, BFS menemukan solusi 13 gerak dengan cost 218, sementara UCS butuh 14 gerak untuk cost yang lebih kecil, yakni 217.
2. **Efisiensi ekspansi.** GBFS secara konsisten menjelajahi simpul paling sedikit. Pada peta besar 30×30 (testcase9), GBFS-H2 hanya butuh 74 iterasi dibanding 311 pada UCS. Trade-off-nya, GBFS-H2 menghasilkan cost 1385—lebih besar 5,5% dari optimal.
3. **Pengaruh kualitas heuristik pada A*.** H3 menghasilkan iterasi paling sedikit (149) karena estimasinya paling ketat. Meski overhead per-node lebih tinggi akibat

Dijkstra internal, total waktu eksekusi A*-H3 (26 ms) tetap lebih cepat dari A*-H2 (31 ms) pada testcase9 berkat penghematan iterasinya.

4. **Kasus tanpa solusi.** Testcase10 memiliki goal yang terkepung lava. Semua algoritma mengeksplorasi tepat 14 simpul yang dapat dijangkau sebelum ruang pencarian habis, membuktikan implementasi menangani kasus ini dengan benar.

BAB V: SOURCE PROGRAM

5.1 Algoritma Pencarian Rute (Pathfinding)

Semua algoritma berbagi struktur `SearchNode` yang menyimpan state saat ini, biaya jalur $g(n)$, nilai evaluasi $f(n)$, serta riwayat gerak. Priority queue diimplementasikan sebagai min-heap kustom agar pemilihan simpul prioritas tertinggi berjalan dalam $O(\log n)$.

Berikut adalah implementasi A* sebagai contoh utama. GBFS hanya berbeda pada baris perhitungan `FCost` (diganti dengan nilai h saja), sedangkan UCS menetapkan `FCost = GCost`.

```
func AStar(puzzle *engine.Puzzle, heuristic HeuristicFunc) (*
    SolveResult, error) {
    start := time.Now()
    initialState := puzzle.InitialState()

    h0, err := heuristic(puzzle, initialState)
    if err != nil {
        return nil, err
    }

    frontier := &MinHeap{}
    frontier.Push(&SearchNode{
        State: initialState, GCost: 0, FCost: h0,
        Moves: []string{}, Slides: []engine.SlideResult{},
    })

    visited := map[engine.State]bool{}
    iterations := 0

    for !frontier.IsEmpty() {
        current := frontier.Pop()
        if visited[current.State] {
            continue
        }
        visited[current.State] = true
        iterations++

        if puzzle.IsGoalState(current.State) {
            return buildResult(current, iterations, time.Since(
                start)), nil
        }

        for _, dir := range searchDirections {
            slide, ok, err := puzzle.Slide(current.State, dir)
            if err != nil {
```



```

        return nil, err
    }
    if !ok || visited[slide.NextState] {
        continue
    }
    nextG := current.GCost + slide.PathCost
    nextH, err := heuristic(puzzle, slide.NextState)
    if err != nil {
        return nil, err
    }
    frontier.Push(&SearchNode{
        State:  slide.NextState,
        GCost:  nextG,
        FCost:  nextG + nextH,
        Moves:  appendMove(current.Moves, dir),
        Slides: appendSlide(current.Slides, slide),
    })
}
}
return notFound(iterations, time.Since(start)), nil
}

```

Algoritma bonus IDA* menggunakan DFS berulang dengan threshold pada nilai $f(n)$. Berikut fungsi rekursifnya:

```

func idaStarDFS(
    puzzle *engine.Puzzle, state engine.State,
    gCost, threshold int, heuristic HeuristicFunc,
    pathVisited map[engine.State]bool,
    moves []string, slides []engine.SlideResult,
) (*SolveResult, int, int, error) {
    h, err := heuristic(puzzle, state)
    if err != nil {
        return nil, 0, 0, err
    }
    fCost := gCost + h
    if fCost > threshold {
        return nil, fCost, 0, nil
    }
    if puzzle.IsGoalState(state) {
        return &SolveResult{
            Found: true, Moves: moves,
            Slides: slides, TotalCost: gCost,
        }, 0, 1, nil
    }

    minNext := unreachableHeuristic
    iterCount := 1

    for _, dir := range searchDirections {
        slide, ok, err := puzzle.Slide(state, dir)
        if err != nil {
            return nil, 0, iterCount, err
        }
        if !ok || pathVisited[slide.NextState] {
            continue
        }
    }
}

```

```

    }
    pathVisited[slide.NextState] = true
    childResult, childNext, childIter, err := idaStarDFS(
        puzzle, slide.NextState, gCost+slide.PathCost,
        threshold,
        heuristic, pathVisited,
        appendMove(moves, dir), appendSlide(slides, slide),
    )
    iterCount += childIter
    pathVisited[slide.NextState] = false
    if err != nil {
        return nil, 0, iterCount, err
    }
    if childResult != nil {
        return childResult, 0, iterCount, nil
    }
    if childNext < minNext {
        minNext = childNext
    }
}
return nil, minNext, iterCount, nil
}

```

5.2 Fungsi Heuristik

Ketiga heuristik menggunakan antarmuka fungsi `HeuristicFunc` yang sama sehingga dapat dipakai secara seragam oleh A*, GBFS, dan IDA*.

H1 — Jarak Manhattan:

```

func HeuristicManhattan(puzzle *engine.Puzzle, state engine.
State) (int, error) {
    if err := validateHeuristicInput(puzzle, state); err != nil
    {
        return 0, err
    }
    current := state.Actor
    total := 0
    for idx := state.NextCheckpoint; idx < puzzle.
CheckpointCount(); idx++ {
        target := puzzle.CheckpointOrder[idx]
        total += manhattanDistance(current, target)
        current = target
    }
    total += manhattanDistance(current, puzzle.Goal)
    return total * puzzle.MinWalkableCost, nil
}

```

H2 — Minimum Slide (BFS ringan ke target berikutnya):

```

func HeuristicMinimumSlide(puzzle *engine.Puzzle, state engine.
State) (int, error) {
    if err := validateHeuristicInput(puzzle, state); err != nil
    {
        return 0, err
    }
}

```

```

    }
    if targetSatisfied(puzzle, state, state) {
        return 0, nil
    }
    type bfsNode struct {
        state engine.State
        depth int
    }
    queue := []bfsNode{{state: state, depth: 0}}
    visited := map[engine.State]struct{}{state: {}}
    for len(queue) > 0 {
        current := queue[0]
        queue = queue[1:]
        for _, direction := range heuristicDirections {
            result, ok, err := puzzle.Slide(current.state,
                direction)
            if err != nil {
                return 0, err
            }
            if !ok {
                continue
            }
            nextState := result.NextState
            if targetSatisfied(puzzle, state, nextState) {
                return (current.depth + 1) * puzzle.
                    MinWalkableCost, nil
            }
            if _, exists := visited[nextState]; exists {
                continue
            }
            visited[nextState] = struct{}{}
            queue = append(queue, bfsNode{state: nextState,
                depth: current.depth + 1})
        }
    }
    return unreachableHeuristic, nil
}

```

H3 — Dijkstra Lokal ke target berikutnya menggunakan priority queue dan menyimpan hasilnya ke dalam cache per-puzzle agar tidak dihitung ulang untuk state yang sama. Implementasi lengkapnya tersedia di `src/backend/algorithm/heuristic.go`.

BAB VI: PENUTUP

6.1 Kesimpulan

Pemodelan permasalahan *Ice Sliding Puzzle* telah berhasil ditranslasikan ke dalam struktur pencarian berbasis ruang status yang memperhitungkan mekanika pergerakan tidak terinterupsi. Dari segi instrumen algoritma, A* terbukti sebagai solusi paling ideal karena mampu menjamin rute terpendek dengan meminimalisir ekspansi buta layaknya UCS, hal ini didukung oleh penggunaan heuristik $h(n)$ yang kuat dan terbukti *admissible*.

6.2 Saran

Berdasarkan hasil eksperimen, metode penyimpanan *cache* terdistribusi dapat ditambahkan agar evaluasi fungsi heuristik yang repetitif (seperti fungsi H3) dapat berjalan lebih singkat pada matriks peta berskala masif.

DAFTAR PUSTAKA

- [1] R. Munir, “Penentuan rute (Route/Path Planning) - Bagian 1,” Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026. [Daring]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.html>
- [2] R. Munir, “Penentuan rute (Route/Path Planning) - Bagian 2,” Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026. [Daring]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.html>
- [3] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985. [Daring]. Tersedia: [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)

LAMPIRAN

Pranala GitHub Repository:

https://github.com/Sakazu01/Tucil3_13524085_13524090

Mekanisme Pembangkitan Suksesor — fungsi Slide (src/backend/engine/movement.go):

```
func (p *Puzzle) Slide(state State, direction string) (
    SlideResult, bool, error) {
    delta, exists := moveDeltas[direction]
    if !exists {
        return SlideResult{}, false, fmt.Errorf("arah_tidak_
            valid:_%q", direction)
    }

    current := state.Actor
    nextCheckpoint := state.NextCheckpoint
    path := make([]Position, 0, p.Rows+p.Cols)
    passedCheckpoints := make([]int, 0, 4)
    totalCost := 0
    moved := false

    for {
        next := current.Add(delta)

        // Batas papan berlaku sama dengan dinding
        if !p.InBounds(next) {
            if !moved {
                return SlideResult{}, false, nil
            }
            return p.buildSlideResult(direction, state, current,
                path, totalCost, passedCheckpoints,
                nextCheckpoint)
        }

        token := p.Cell(next)

        if token == TileWall {
            if !moved {
                return SlideResult{}, false, nil
            }
            return p.buildSlideResult(direction, state, current,
                path, totalCost, passedCheckpoints,
                nextCheckpoint)
        }

        // Lintasan melewati lava selalu ilegal
        if token == TileLava {
            return SlideResult{}, false, nil
        }

        // Checkpoint di luar urutan berlaku sebagai dinding
        checkpoint, isCheckpoint := parseCheckpointToken(token)
        if isCheckpoint && checkpoint > nextCheckpoint {
            if !moved {
```

```

        return SlideResult{}, false, nil
    }
    return p.buildSlideResult(direction, state, current,
        path, totalCost, passedCheckpoints,
        nextCheckpoint)
}

current = next
moved = true
path = append(path, current)
totalCost += p.CostAt(current)

if isCheckpoint && checkpoint == nextCheckpoint {
    passedCheckpoints = append(passedCheckpoints,
        checkpoint)
    nextCheckpoint++
}
}
}

```